Predicting Connect Four Game Outcomes using Machine Learning

Lama Alghamdi Fay Altheeb Lama Aldakheelallh Nouf Bin Jurayyad

Abstract—this study formulates Connect Four game as a machine learning problem using a board of 6×7 grid flattened into 42 features encoded into (1,0, -1) and additional one for turn. After preprocessing the dataset (dividing data and applying feature scaling) and applying feature engineering to add new features for strategic reasons for advanced models, we train baseline classifiers (decision tree, logistic regression, random forest in scikit-learn) and advanced models (Gradient Boosting, XGBoost, MLP, LightGBM, CatBoost). We reserve placeholders for experimental results and confusion matrix figures. Preliminary analysis indicates that CatBoost yield higher accuracy than others.

***Key words*— CatBoost, Decision Tree, Feature Engineering, Feature Encoding, Gradient Boost, LightGBM, Logistic Regression, Machine Learning, Neural Networks, Random Forest, XGBoost**

I. Introduction

Artificial intelligence researchers have traditionally used games as a set of benchmark domains to test their learning algorithms. As a two-player deterministic game, Connect Four is a strategy game on a 7x6 grid. To be solved, the players take turns, where the first player has a forced win with perfect play. The game ends in a tie only when the board fills without a four-in-a-row[1] . Although the game can be mathematically solved, with the first player having the ability to win in case of perfect play, it is computationally expensive to implement such optimal strategies in real-time by use of computationally costly search algorithms.

Classical game-playing AI depends on adversarial search, such as Minimax with Alpha-beta pruning. It is an effective approach, but it is computationally intensive and does not scale well with search depth. Recent successes in machine learning show that supervised learning can be used to approximate optimal gameplay by learning patterns of the game directly by looking at games' data, such as chess [2]

Connect Four poses special problems for supervised learning in predicting outcomes (win, loss, or draw). Unlike traditional board games, they must follow the laws of gravity, that is, fall to the lowest available row within a chosen column. This vertical organization compels models to acquire varying patterns of space, contrasting with games such as chess. Moreover, the small size of the board makes one move significant enough to change the course of the game, and the models should be able to perceive the risk in the near future and the long-term strategy. Although not every column is a move that can be made everywhere, there are only strong moves in one or more board states and many potential moves, leading

to an imbalance in training data [3] .

We are investigating several learning paradigms starting with interpretable baseline models, including logistic regression, and decision tree. Moving onto ensemble algorithms like random forest and gradient boost and more complex such as XGBoost and neural networks. Additionally, we analyze feature engineering measures to perform a more accurate representation of the spatial structure of the game of Connect Four, with the explicit representation of a winning pattern and the recognition of threat.

II. Methodology

A. DATASET DESCRIPTION

The dataset used in this study represents board states of the Connect-4 game, a two-player strategy game played on a 7-column × 6-row vertical grid. Each sample in the dataset corresponds to a single game position, capturing the complete state of the board at a specific point during gameplay.

The game board is represented as a flattened vector of 42 features (*p1 to p42),* ordered in row-major format from bottom-to-top and left-to-right. Each feature encodes the occupancy state of a cell; +1 indicates the current player's piece, -1 represents the opponent's piece, and 0 denotes an empty cell. This representation follows the standard Connect-4 convention, where pieces are subject to gravity and fall to the lowest available position in each column.

An auxiliary feature, *turn*, indicates which player is to move next (+1 for the current player's turn, -1 for the opponent's turn). For the training and validation sets, each sample includes *label_move_col*, representing the optimal column index (0–6) for the next move.

B. PREPROCESSING

The raw data underwent a systematic preprocessing pipeline consisting of multiple sequential steps, all aimed at ensuring data quality and rendering the dataset suitable for subsequent model training.

1. Data Loading and Splitting

The dataset was provided as pre-split CSV files (train.csv, val.csv, test.csv) to facilitate consistent evaluation and minimize the risk of data leakage between training and testing phases. The structure of the dataset is as follows:

- Features (X): 42 board position features (*p1–p42*) along with a turn indicator, representing the current state of the game.
- Target (y): The optimal move column (*label_move_col*) used for training and validation purposes.
- Test set: Features only, with the corresponding labels withheld to enable blind evaluation of model performance.

This pre-defined partitioning ensures that experiments can be reliably reproduced, and results compared across different models. By keeping the training and test sets separate, it allows for a fair evaluation of model performance and provides a solid foundation for the subsequent preprocessing and training steps.

2. Feature scaling

Although the original features are bounded within {-1, 0, +1}, standardization was applied to normalize feature distributions. The scikit-learn StandardScaler transformed all features to zero mean and unit variance using the formula below:

$$z = (x - \mu) / \sigma \quad (1)$$

where x is the original feature value, μ is the training set mean, σ is the training set standard deviation, and z is the standardized value.

The significance of this transformation is that some machine learning algorithms are sensitive to feature scales and operate best when features are normalized, such as gradient-based optimization algorithms (logistic regression and neural networks). In this research, we used the common procedure of using the scaler only on the training data and then learned a transformation on the validation and test sets without leakage.

Once these preprocessing steps have been done, the data is converted into a uniform, normalized format that could be used in feature engineering and machine learning model training.

C. FEATURE ENGINEERING

Although the original 43 features give full information about the board state, we needed to add an additional number of features in order to reflect strategic patterns and tactical opportunities. Our feature engineering aims at eliciting domain-specific knowledge in the game of Connect Four.

Analysis of expert Connect 4 game highlights two key aspects: strategy and tactics[1] . Strategy involves long-term planning, such as building patterns and controlling important areas over several moves. Tactics focus on immediate threats and opportunities that can determine the game in just one or two moves. This distinction guided the feature design, leading to the inclusion of both slow-building positional features and urgent tactical features [1]

Feature Categories

Game State Context: Essential data regarding the current state of the board is conveyed through several key measures. This includes a count of the pieces for each player and an assessment of the material advantage, which reflects the difference in pieces between the current player and their opponent. The board fill ratio is analyzed to understand the stage of the game, indicating the degree of saturation on the board.
Additionally, spatial availability is captured through both vertical and horizontal dimensions. Column availability is monitored to identify the remaining legal moves, and the heights of these columns are recorded to determine their limiting heights. While empty spaces distribution is tracked across each of the six horizontal rows, revealing vertical congestion patterns, which aids the model in grasping the overall phase of the game and the available strategic space for maneuvering.

Features of Positional Control: Strategic positioning on the board can be assessed through several key aspects. One of the most critical is the control of the center column, as it provides maximum connectivity for creating winning combinations. Additionally, edge control in the left and right columns is important because it allows players to limit their opponents' options and establish robust defensive formations. Control of the bottom row is also vital, as it serves as the foundation for vertical builds and influences the stability of future moves. These elements reflect the spatial strategic principles that professional players intuitively understand.

Pattern Recognition and Threat Detection Characteristics: The system applies all directions of piece sequences in all conceivable directions, horizontal, vertical, and diagonal. The patterns of two_in_a_row are discovered to identify the existing setups in the early game and any incremental prospects. There are critical tactical scenarios that are recorded based on a systematic study of sequences in which a player possesses three consecutive pieces with only one vacancy left. These threat identification functions are computed in a symmetrical way between both the player and the opponent, and as a result, the model can concurrently find offensive opportunities and defensive needs. The difference between the short-term tactical danger and the long-term strategic trends assists the model in prioritizing the moves accordingly.

Critical Move Features: Simulation-based analysis is done to implement features that identify game-deciding moves. The system then determines in each column available whether the current player would win immediately after placing the piece in the column. Equally, it recognizes columns, in which the opponent might win next turn, prioritizing pressing defensive needs. The cumulative number of winning moves that each player has will be counted, which is a measure of offensive strength and defensive weakness.

Advanced Tactical Features: Complex game theoretical ideas are represented by the detection of moves that give rise to several simultaneous winning positions. The system recognizes the moves that make two or more winning threats at the same time, tactical forks that indicate impassable positions that will ensure victory despite the opponent. A detailed board scoring system analyzes the overall position by rating all possible four-cell windows, and a window is rated with an aggregate score based on the distribution of pieces. Diagonal windows are given a heavier weight because of their strategic hard-to-defend position. Also, a column preference score is used to weight pieces based on their strategic value, with the column that is in the center having a greater weight than the one located at an edge.

D. MODELS USED

Baseline Models:
For baseline comparison, we implement several classical classifiers using scikit-learn. A Decision Tree classifier [4] is trained to directly partition the feature space; it is prone to overfitting but provides interpretable rules. A Logistic Regression model [5] is used to establish a linear decision boundary. We also train a Random Forest [6ensemble, which averages many decision trees to improve generalization. Model performance is evaluated using accuracy. In addition, we generated confusion matrices for each model on the validation set. These baselines serve as reference points.

Advanced Models:
After implementing baseline models, we are looking for better results in more advanced algorithms that are able to model more complex, non-linear relationships between features. Starting with Gradient Boost, which is a form of ensemble learning, a powerful predictor model built by the sequential combination of weak learners, with each additional tree, the residual errors of its predecessors are fixed using gradient descent learning [7]

On the other hand, XGBoost builds upon gradient boosting with L1 and L2 regularization, sparse features, parallelization of tree building, and tree pruning optimization; we optimized XGBoost on structured data to maximize its performance and to obtain feature importance metrics that are interpretable[8] .
Additionally, we apply a Multi-Layer Perceptron (MLP) neural network (including fully connected layers, Rectified Linear Unit (ReLU) activation, and dropout regularization) to train hierarchical strategic pattern representations. Different architectural designs were investigated with different numbers of hidden layers, units per layer, and dropout rates. The network output used SoftMax activation to generate probability distributions over seven target classes, and the Adam optimizer, cross-entropy loss, and early stopping were used to reduce overfitting [9] .
The other phase consisted of the implementation of Light Gradient Boosting Machine (LightGBM) using a histogram-based learning paradigm and a leaf-wise tree growth strategy, as opposed to the level-wise expansion of the traditional paradigm. It is more accurate with fewer trials, and has incredible computational efficiency, and allows massive hyperparameter optimization, learning rate, number of leaves, maximum depth, and early stopping according to validation metrics [10] .

Finally, we implemented CatBoost (Categorical Boosting), a gradient boosting framework that addresses prediction shift and overfitting through ordered boosting and a novel algorithm for processing categorical features[11] . CatBoost's ordered target statistics prevent target leakage during training, while its symmetric tree structure ensures faster prediction times. The model was trained using MultiClass loss function with GPU-based task processing, leveraging parallel computation for accelerated training. Early stopping with 100-round patience was employed to prevent overfitting by monitoring evaluation set accuracy and achieved competitive performance on the Connect-4 classification task.

III. EXPERIMENTAL RESULTS

Multiple machine learning models were evaluated to predict the next move in the Connect Four game dataset. Initially, the models were trained and validated using baseline features that represented the board configuration in its simplest form. Subsequently, the models were enhanced with engineered features designed to capture deeper strategic and tactical opportunities. The performance outcomes of these models are summarized in Table 1 and confusion matrices in figures 1-8.

Table 1: Accuracy rate on all the supervised ML models used

Model	*Validation Accuracy*
Logistic Regression	*40.2%*
Decision Tree	*47.1%*
Random Forest	*58%*
Gradient Boosting	*68.25%*
Neural Network (MLP)	*63.97%*
XGBoost	*70.89%*
LightGBM	*70.69%*
CatBoost	71.15%

Table 1 presents the validation accuracy for all seven models tested. XGBoost achieved the highest accuracy at 72.39%, followed closely by LightGBM at 72% and the Neural Network (MLP) at 70%. Among ensemble methods, Gradient Boosting reached 62% and Random Forest 58%. Simpler models showed lower performance, with Decision Tree at 47.1% and Logistic Regression at 40.2%.

Confusion matrices for each model are shown in Figures 1-8, illustrating the distribution of predicted versus actual column choices for each move.

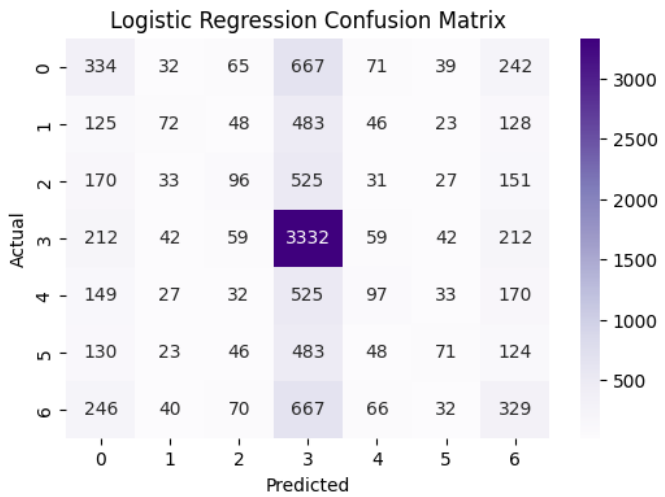

Figure1. Confusion Matrix for Logistic Regression

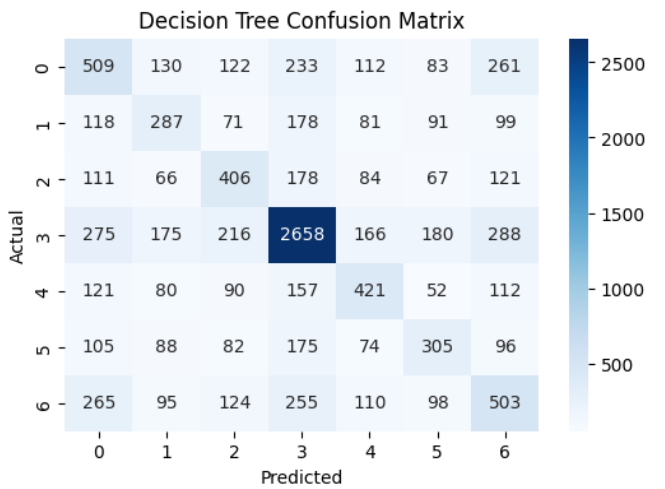

Figure2. Confusion Matrix for Decision Tree

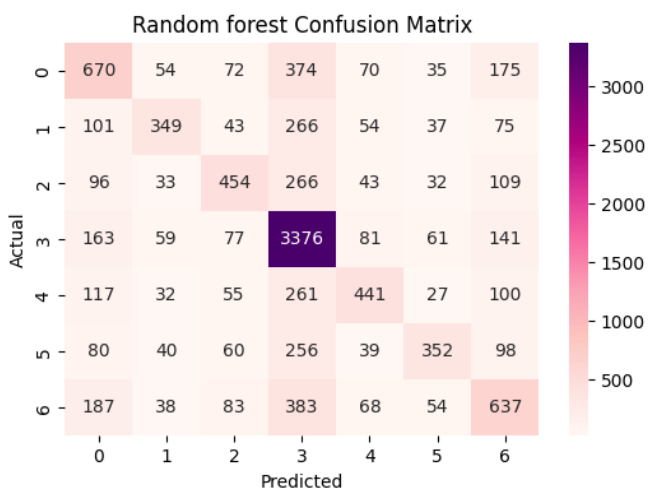

Figure3. Confusion Matric for Random Forest

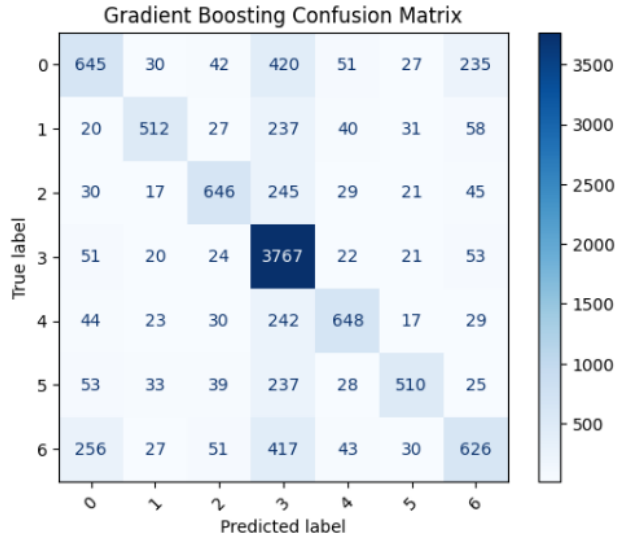

Figure4. Confusion Matrix for Gradient Boosting

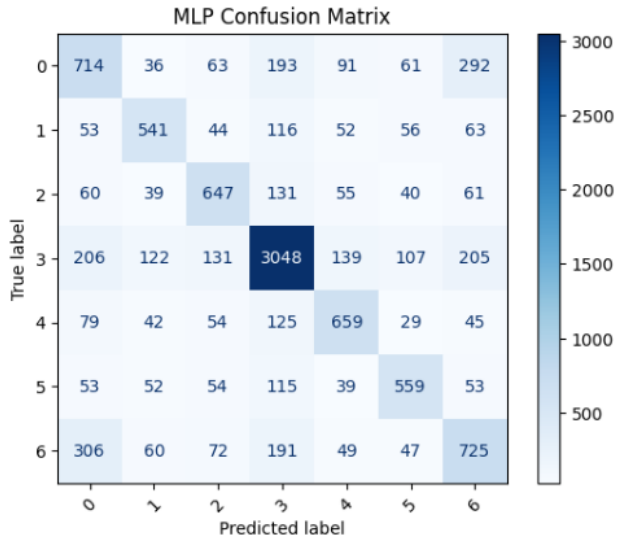

Figure5. Confusion Matrix for Neural Network (MLP)

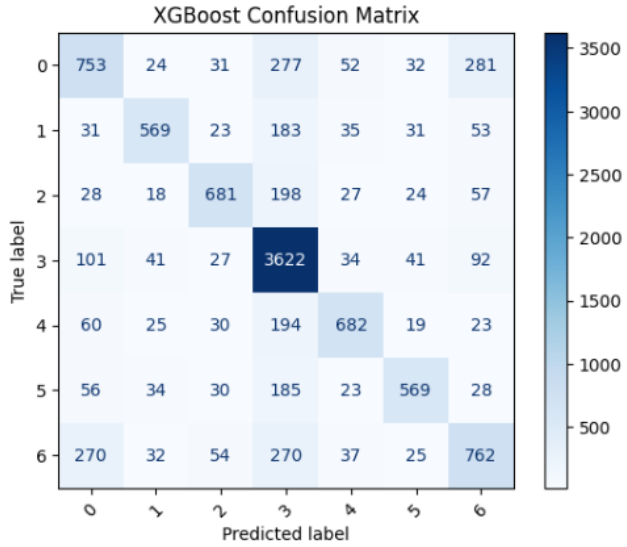

Figure6. Confusion Matrix for XGBoost

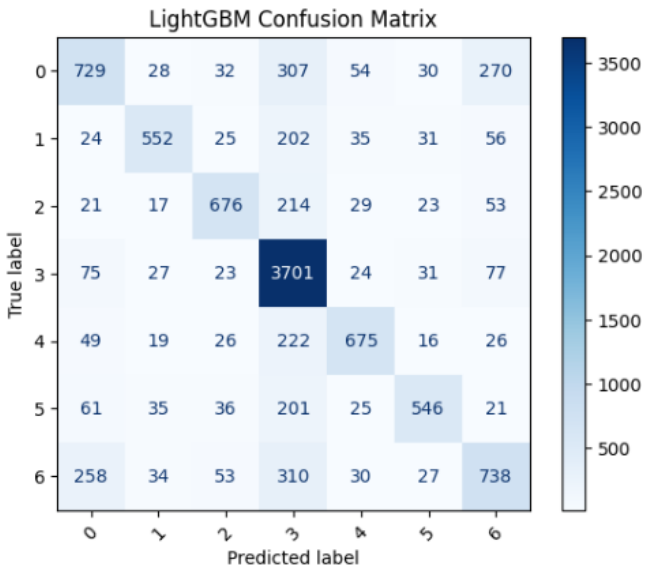

Figure7. Confusion Matrix for LightGBM

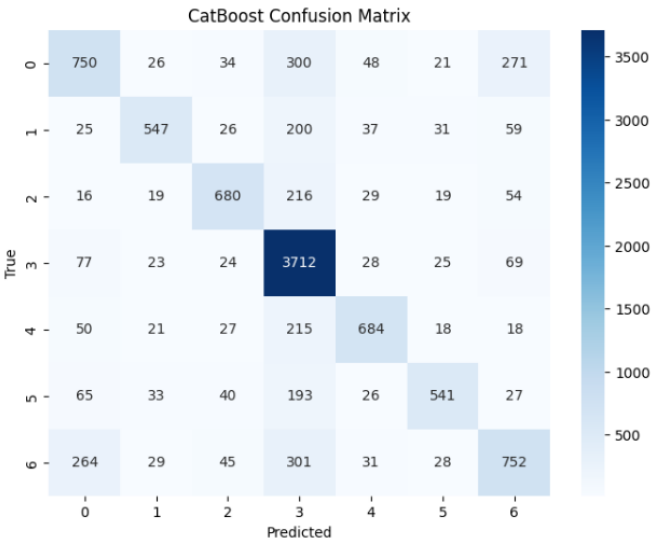

Figure8. Confusion Matrix for CatBoost

IV. TUNING

Hyperparameter tuning is a critical step in machine learning that involves systematically adjusting model parameters to achieve optimal performance. Unlike model parameters that are learned from data during training, hyperparameters are set before the training process begins and significantly influence the model's ability to generalize to unseen data. For tree-based ensemble methods like CatBoost, proper tuning can substantially improve prediction accuracy and prevent overfitting.

The CatBoost classifier underwent systematic hyperparameter tuning to optimize performance on the Connect-4 classification task. The initial experiments explored different hyperparameter search space included iterations (500, 800), learning rate (0.03, 0.05), tree depth (6, 8), and L2 leaf regularization (3, 5). RandomizedSearchCV performed 10 iterations with 3-fold cross-validation to identify the optimal parameter combination. This approach efficiently explored the hyperparameter space while avoiding exhaustive grid search, balancing computational cost with thorough exploration. The best parameters identified through this process were then used to train the final CatBoost model with GPU acceleration for faster computation. Early stopping rounds were configured to 100, automatically halting training when evaluation metrics plateaued, thus preventing unnecessary computation and overfitting [11] . This systematic tuning approach ensured that the model achieved strong performance while maintaining generalization capability.

V. DISCUSSION

A total of eight models were trained on a Connect Four game dataset to predict a player's next move. Among the three simple models, Random Forest outperformed Logistic regression and Decision Tree with a validation accuracy rate of 58%. It is plausible that the linear assumption of Logistic Regression and the simplicity of the Decision Tree limited their ability to capture complex strategic patterns of the Connect Four game. It is worth noting that simple models were trained before applying feature engineering; therefore, they had fewer complex data to work with, compared to the advanced models.

As for the validation results with the advanced models, a CatBoost classifier obtained a score of 71.15%, outperforming all seven models in this study. The second highest score was achieved by the XGBoost and LightGBM classifiers, receiving a score of 70.89% and 70.69%, respectively, with XGBoost exceeding by a small margin. A Gradient Boosting model with 68.25% was the lowest tree-based model among advanced models, but it successfully obtained a higher score than a Neural Network (MLP), which achieved 63.97%. Nonetheless, all those models presented a significant improvement against previous simpler models. Although obtaining one of the highest score at first, XGBoost's accuracy score drops heavily when applied to a testing set, reaching approximately 20% of test score. This drastic change may have been the result of extensive feature engineering and board-flipping data augmentation, which led the models to overfit, memorizing patterns from data they had seen, and generalizing badly to new data. As a result, data augmentation was eliminated from the process, and through a number of trails, a new set of extracted features were chosen. This is also where the CatBoost classifier was introduced, and it demonstrated stronger generalization to the test set than XGBoost, achieving a test accuracy score of 69.8%. The CatBoost classifier seemed to help overcome the overfitting issue, potentially due to many of its advantages, as it learns slowly using boosting, and uses strong regularization. In addition, the classifier was trained with hyperparameters tuning, which can be effective in increasing accuracy and reducing overfitting. The CatBoost model appeared to be the most generalizable model in this study.

VI. CONCLUSION

This project compared different Machine Learning models, with various natures and capabilities, to find the best model that

predicts the next move of a player during a Connect Four game. The process for finding the highest accuracy rate took several steps, including data preprocessing, feature engineering, experimentation with models, and evaluating performance. The results showed that the ensemble models outperform simple models. Although XGBoosting and LightGBM experienced overfitting issues, CatBoots model successfully overcomes the problem, yielding remarkable outcomes. Thos establish it to be the best performing among other models. Overall, the results indicate the significant role of quality feature engineering and model selection in obtaining high-accuracy gameplay prediction.

APPENDIX

```
 import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pickle
import warnings
from sklearn.model_selection import train_test_split,
RandomizedSearchCV
from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report
warnings.filterwarnings('ignore')
# Install CatBoost Model #
try:
    from catboost import CatBoostClassifier
except ImportError:
    import subprocess
    subprocess.check_call(['pip', 'install', 'catboost'])
    from catboost import CatBoostClassifier


# Data Loading #
train_df = pd.read_csv('/content/sample_data/train.csv')
val_df   = pd.read_csv('/content/sample_data/val.csv')
test_df  = pd.read_csv('/content/sample_data/test.csv')


# Feature Engineering #
def check_win(board, player):
    # Horizontal
    for r in range(6):
        for c in range(4):
            if np.all(board[r, c:c+4] == player):
                return True
    # Vertical
    for c in range(7):
        for r in range(3):
            if np.all(board[r:r+4, c] == player):
                return True
    # Diagonal down-right
    for r in range(3):
        for c in range(4):
            if all(board[r+i, c+i] == player for i in range(4)):
                return True
    # Diagonal down-left
    for r in range(3):
        for c in range(3, 7):
            if all(board[r+i, c-i] == player for i in range(4)):
                return True
    return False

def simulate_drop_and_get(board_state, col_idx, player):
    b = board_state.copy()
    for r in range(6):
        if b[r, col_idx] == 0:
            b[r, col_idx] = player
            return b
    return None

def count_winning_moves(board_state, player):
    cnt = 0
    for c in range(7):
        newb = simulate_drop_and_get(board_state, c, player)
        if newb is not None and check_win(newb, player):
            cnt += 1
    return cnt

def window_score_board(board_state):
    score = 0.0
    # Horizontal
    for r in range(6):
        for c in range(4):
            window = board_state[r, c:c+4]
            score += (window==1).sum()*2.0 - (window==-
1).sum()*2.0
    # Vertical
    for c in range(7):
        for r in range(3):
            window = board_state[r:r+4, c]
            score += (window==1).sum()*2.0 - (window==-
1).sum()*2.0
    # Diagonals
    for r in range(3):
        for c in range(4):
            window = np.array([board_state[r+i, c+i] for i in
range(4)])
            score += (window==1).sum()*2.5 - (window==-
1).sum()*2.5
    for r in range(3):
        for c in range(3,7):
            window = np.array([board_state[r+i, c-i] for i in
range(4)])
            score += (window==1).sum()*2.5 - (window==-
1).sum()*2.5
    return score

def engineer_features(df):
    board_cols = [f'p{i}' for i in range(1, 43)]
    board = df[board_cols].values
    board_3d = board.reshape(-1, 6, 7)

    features = {}
    n = len(df)

    # Original board positions and turn
    for col in board_cols:
```

```
        features[col] = df[col].values
    features['turn'] = df['turn'].values

    # Column heights & availability
    for col_idx in range(7):
        column = board_3d[:, :, col_idx]
        heights = (column != 0).sum(axis=1)
        features[f'col_{col_idx}_height'] = heights
        available = (column[:, -1] == 0).astype(int)
        features[f'col_{col_idx}_available'] = available

    # Piece counts
    current_player_pieces = (board == 1).sum(axis=1)
    opponent_pieces = (board == -1).sum(axis=1)
    features['current_player_count'] = current_player_pieces
    features['opponent_count'] = opponent_pieces
    features['piece_difference'] = current_player_pieces -
opponent_pieces

    # Center control
    center_col = board_3d[:, :, 3]
    features['center_current_player'] = (center_col ==
1).sum(axis=1)
    features['center_opponent'] = (center_col == -
1).sum(axis=1)

    # Horizontal & Vertical threats
    threats_current = np.zeros(n)
    threats_opponent = np.zeros(n)
    for i in range(n):
        board_state = board_3d[i]
        for row in range(6):
            for col in range(4):
                window = board_state[row, col:col+4]
                if (window == 1).sum() == 3 and (window ==
0).sum() == 1:
                    threats_current[i] += 1
                if (window == -1).sum() == 3 and (window ==
0).sum() == 1:
                    threats_opponent[i] += 1
        for col in range(7):
            for row in range(3):
                window = board_state[row:row+4, col]
                if (window == 1).sum() == 3 and (window ==
0).sum() == 1:
                    threats_current[i] += 1
                if (window == -1).sum() == 3 and (window ==
0).sum() == 1:
                    threats_opponent[i] += 1

    features['threats_current_player'] = threats_current
    features['threats_opponent'] = threats_opponent

    # Diagonal threats
    diag_threats_current = np.zeros(n)
    diag_threats_opponent = np.zeros(n)
    for i in range(n):
        board_state = board_3d[i]
        for row in range(3):
            for col in range(4):
                window = [board_state[row+j, col+j] for j in
range(4)]
                if window.count(1) == 3 and window.count(0) == 1:
                    diag_threats_current[i] += 1
                if window.count(-1) == 3 and window.count(0) ==
1:
                    diag_threats_opponent[i] += 1
        for row in range(3):
            for col in range(3, 7):
                window = [board_state[row+j, col-j] for j in
range(4)]
                if window.count(1) == 3 and window.count(0) == 1:
                    diag_threats_current[i] += 1
                if window.count(-1) == 3 and window.count(0) ==
1:
                    diag_threats_opponent[i] += 1

    features['diag_threats_current'] = diag_threats_current
    features['diag_threats_opponent'] = diag_threats_opponent

    # Two-in-a-row patterns
    two_in_row_current = np.zeros(n)
    two_in_row_opponent = np.zeros(n)
    for i in range(n):
        board_state = board_3d[i]
        for row in range(6):
            for col in range(6):
                if board_state[row, col] == board_state[row, col+1]
!= 0:
                    if board_state[row, col] == 1:
                        two_in_row_current[i] += 1
                    else:
                        two_in_row_opponent[i] += 1
        for col in range(7):
            for row in range(5):
                if board_state[row, col] == board_state[row+1, col]
!= 0:
                    if board_state[row, col] == 1:
                        two_in_row_current[i] += 1
                    else:
                        two_in_row_opponent[i] += 1

    features['two_in_row_current'] = two_in_row_current
    features['two_in_row_opponent'] = two_in_row_opponent

    # Edge control & bottom row
    left_edge = board_3d[:, :, 0]
    right_edge = board_3d[:, :, 6]
    features['left_edge_current'] = (left_edge == 1).sum(axis=1)
    features['left_edge_opponent'] = (left_edge == -
1).sum(axis=1)
    features['right_edge_current'] = (right_edge ==
1).sum(axis=1)
    features['right_edge_opponent'] = (right_edge == -
1).sum(axis=1)
    bottom_row = board_3d[:, 0, :]
    features['bottom_current'] = (bottom_row ==
1).sum(axis=1)
    features['bottom_opponent'] = (bottom_row == -
1).sum(axis=1)
```

```
    # Board fill ratio
    features['board_fill'] = (board != 0).sum(axis=1) / 42.0

    # Empty spaces per row
    for row_idx in range(6):
        row = board_3d[:, row_idx, :]
        features[f'row_{row_idx}_empty'] = (row ==
0).sum(axis=1)

    # Strategic features: immediate wins, double threats
    for col_idx in range(7):
        features[f'col_{col_idx}_immediate_win'] = np.zeros(n,
dtype=int)
        features[f'col_{col_idx}_opponent_immediate_win'] =
np.zeros(n, dtype=int)
        features[f'col_{col_idx}_double_threat'] = np.zeros(n,
dtype=int)

    features['any_immediate_win'] = np.zeros(n, dtype=int)
    features['any_opponent_immediate_win'] = np.zeros(n,
dtype=int)
    features['num_current_winning_moves'] = np.zeros(n,
dtype=int)
    features['num_opponent_winning_moves'] = np.zeros(n,
dtype=int)
    features['num_double_threats'] = np.zeros(n, dtype=int)
    features['window_score'] = np.zeros(n, dtype=float)
    features['column_preference_score'] = np.zeros(n,
dtype=float)

    col_weights = np.array([3 - abs(c - 3) for c in range(7)])

    for i in range(n):
        board_state = board_3d[i]
        features['window_score'][i] =
window_score_board(board_state)

        double_threat_count = 0
        any_im_win = 0
        any_opp_im_win = 0
        col_pref_score = 0.0

        for c in range(7):
            newb_cur = simulate_drop_and_get(board_state, c, 1)
            if newb_cur is not None:
                if check_win(newb_cur, 1):
                    features[f'col_{c}_immediate_win'][i] = 1
                    any_im_win = 1
                wins_after = count_winning_moves(newb_cur, 1)
                if wins_after >= 2:
                    features[f'col_{c}_double_threat'][i] = 1
                    double_threat_count += 1

            newb_opp = simulate_drop_and_get(board_state, c, -1)
            if newb_opp is not None:
                if check_win(newb_opp, -1):
                    features[f'col_{c}_opponent_immediate_win'][i]
= 1
                    any_opp_im_win = 1
            col_pref_score += col_weights[c] * (board_state[:, c]
== 1).sum()

        features['num_current_winning_moves'][i] =
count_winning_moves(board_state, 1)
        features['num_opponent_winning_moves'][i] =
count_winning_moves(board_state, -1)
        features['any_immediate_win'][i] = any_im_win
        features['any_opponent_immediate_win'][i] =
any_opp_im_win
        features['num_double_threats'][i] = double_threat_count
        features['column_preference_score'][i] = col_pref_score

    return pd.DataFrame(features)

#  Apply features #

X_train = engineer_features(train_df)
y_train = train_df['label_move_col'].values

X_val = engineer_features(val_df)
y_val = val_df['label_move_col'].values

X_test = engineer_features(test_df)
# Hyperparameters Tuning #
param_distributions = {
    'iterations': [1000, 1500, 2000],
    'learning_rate': [0.01, 0.03, 0.05],
    'depth': [7, 9, 11],
    'l2_leaf_reg': [1, 3, 5],
    'border_count': [32, 64, 128]
}

base_model = CatBoostClassifier(
    loss_function='MultiClass',
    eval_metric='Accuracy',
    random_seed=42,
    verbose=0
)

random_search = RandomizedSearchCV(
    base_model,
    param_distributions=param_distributions,
    n_iter=10,
    cv=3,
    scoring='accuracy',
    random_state=42,
    n_jobs=-1
)

random_search.fit(X_train, y_train)

best_params = random_search.best_params_
print("Best Params:", best_params)

# Model Training #
cat_model = CatBoostClassifier(
    **best_params,
    loss_function='MultiClass',
```

```
    eval_metric='Accuracy',
    random_seed=42,
    verbose=0,
    early_stopping_rounds=100
)

cat_model.fit(X_train, y_train, eval_set=(X_val, y_val),
verbose=False)
# Model Evaluation #
val_pred = cat_model.predict(X_val).reshape(-1).astype(int)
val_acc = accuracy_score(y_val, val_pred)
cm = confusion_matrix(y_val, val_pred)

print(f"Validation Accuracy: {val_acc:.4f}")

print(classification_report(
    y_val, val_pred,
    target_names=[f'Col_{i}' for i in range(7)],
    digits=4
))


# Visualization #

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Validation Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()
```

VII. References

[1] V. Allis and Department of Mathematics and Computer Science, Vrije Universiteit, Amsterdam, The Netherlands, "A Knowledge-based Approach of Connect-Four: The Game is Solved: White Wins," thesis, Oct. 1988. [Online]. Available: https://rainerrosenberger.de/x_spiele/download/connect4_victor_allis.pdf

[2] D. Silver *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, Dec. 2018, doi: https://doi.org/10.1126/science.aar6404.

[3] H. He and E. A. Garcia, "Learning from Imbalanced Data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009, doi: https://doi.org/10.1109/tkde.2008.239.

[4] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification And Regression Trees*. Routledge, 2017. doi: https://doi.org/10.1201/9781315139470.

[5] "Applied Logistic Regression," *Google Books*, 2025. https://books.google.com.sa/books?hl=en&lr=&id=bRoxQBIZRd4C&oi=fnd&pg=PR13&dq=+D.+W.+Hosmer+Jr. (accessed Nov. 01, 2025).

[6] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[7] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[8] T. Chen and C. Guestrin, "XGBoost: a Scalable Tree Boosting System," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD '16*, vol. 1, no. 1, pp. 785–794, Aug. 2016, doi: https://doi.org/10.1145/2939672.2939785.

[9] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015, doi: https://doi.org/10.1038/nature14539.

[10] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *Advances in Neural Information Processing Systems*, vol. 30, 2017, Available:

[11] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, Yandex, and Moscow Institute of Physics and Technology, "CatBoost: unbiased boosting with categorical features," *arXiv*, Jan. 2019, [Online]. Available: https://arxiv.org/abs/1706.09516v5